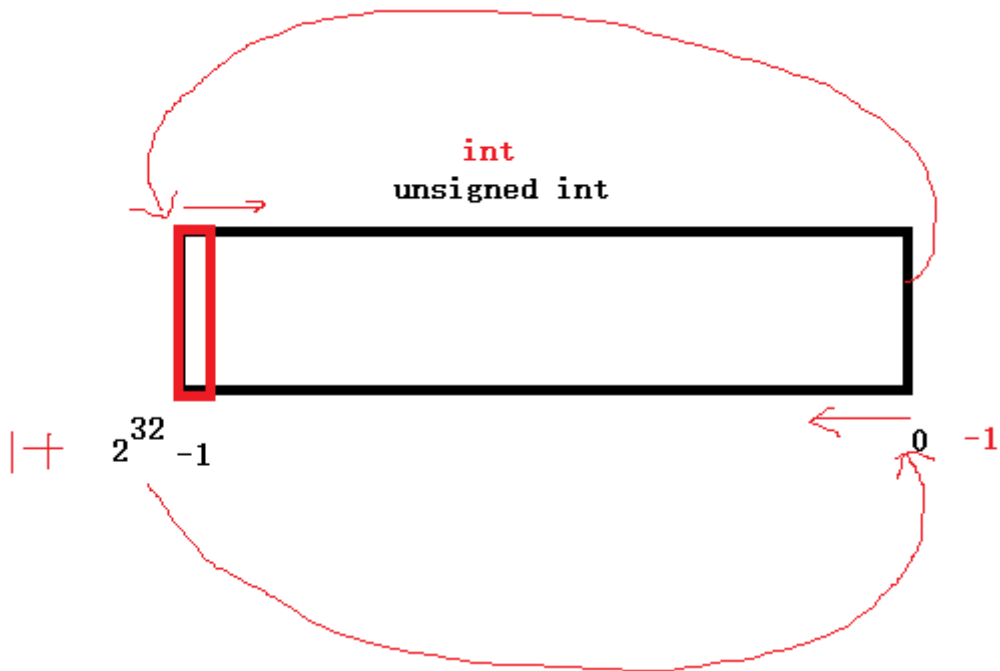


数值溢出

- 如果 int 类型变量，已经取最大值 (2147483647)，再给这个变量 +1。就发了溢出。
 - 上溢出：最大值+1
 - 下溢出：最小值-1



无符号数

- 取值范围 $0 \sim 2^{32}-1$
- 最大值 + 1 $\longrightarrow 0$
- $0-1 \longrightarrow$ 最大值。

```
1 // 上溢出
2 int main(void)
3 {
4     unsigned int a = UINT_MAX; // 取出无符号最大值
5
6     printf("最大值+1 = %d\n", a + 1); // 0
7     printf("最大值+2 = %d\n", a + 2); // 1
8
9     return 0;
10 }
11 // 下溢出
```

```
// 下溢出。
int main(void)
{
    unsigned int a = 0; // 取出无符号最小值

    printf("最小值-1 = %u, 最大值 = %u\n", a - 1, UINT_MAX);

    return 0;
}
```

选择Microsoft Visual Studio 调试控制台

最小值-1 = 4294967295, 最大值 = 4294967295

有符号数

- 上溢出
 - 最大值+1, 上溢出 == 最小值。

```
1 int main(void)
2 {
3     int a = INT_MAX; // 取出无符号最小值
4
5     printf("最大值+1 = %d, 最小值 = %d\n", a+1, INT_MIN);
6
7     return 0;
8 }
```

```
// signed 上溢出。
int main(void)
{
    int a = INT_MAX; // 取出无符号最小值

    printf("最大值+1 = %d, 最小值 = %d\n", a+1, INT_MIN);

    return 0;
}
```

选择Microsoft Visual Studio 调试控制台

最大值+1 = -2147483648, 最小值 = -2147483648

- 下溢出
 - 最小值-1, 下溢出 == 最大值。

```
// signed 下溢出。
int main(void)
{
    int a = INT_MIN; // 取出有符号最小值

    printf("最小值-1 = %d, 最大值 = %d\n", a - 1, INT_MAX);

    return 0;
}
```

选择Microsoft Visual Studio 调试控制台

最小值-1 = 2147483647, 最大值 = 2147483647

小结

- 大部分编译器，对于数值溢出，采用上述处理方式。但，个别编译器有可能不能。
- 受数值溢出影响，进行数据存储时，要选择恰当的数据类型。389482390483209 --- int

不常用关键字（了解）

- extern:
 - 表示声明。声明没有内存空间。不能提升为定义。
- const:
 - 限制一个变量为只读。—— 常量。
- Volatile:
 - 防止编译优化。
- register:
 - 定义一个寄存器变量。没有内存地址。

输入输出函数

输出函数

- printf();
 - %d、%c、%x、%u、%s
- putchar() 函数:
 - 输出一个字符到屏幕。
 - 'abcZ' 是错误的定义，既不是 char 也不是字符串。

```
1 putchar(97);    // 97 == 'a'
2
3 putchar('\n');  // 输出换行符。
4
5 putchar('b');
6
7 putchar('\n');  // 使用率较高。 printf("\n");
8
9 putchar('abcZ'); // 'abcZ' 是错误的定义，既不是 char 也不是字符串。
```

输入函数

scanf 函数

使用

- 安装指定个格式匹配符，获取指定类型数据。
- 获取整数

- ```
1 int a; // 可以定义a变量（有内存空间），也可以声明（自动提升成定义，有内存空间）
2 scanf("%d", &a); // &: 取变量a的地址 —> 拿到a的内存空间。
```

- 示例:

- ```

1 // scanf 输入
2 int main(void)
3 {
4     //int a = 10;
5     int a;    // 在scanf执行时, 会提示为定义。
6
7     int ret = scanf("%d", &a); //scanf可以从键盘获取用户输入, 用户根据
    %d 输入整数。
8
9     printf("获取的a为: %d\n", a);
10
11     return EXIT_SUCCESS;
12 }
```

- 一次性获取多个整数
- 常见的错误书写方法

```

1 int a, b, c;    // 一次性创建多个 int 类型变量。
2
3 // scanf 可以从键盘获取用户输入, 用户根据 多个%d 输入整数。
4 int ret = scanf("%d%d%d", &a, &b, &c);
5
6 // 上述写法编译器无法识别, 输入的 123 应该怎样分配给 3 个 %d
```

- 正确的书写方式: (推荐)

```

1 int main(void)
2 {
3     int a, b, c;    // 一次性创建多个 int 类型变量。
4
5     // scanf 可以从键盘获取用户输入, 用户根据 多个%d 输入整数。
6     int ret = scanf("%d %d %d", &a, &b, &c);
7
8     printf("获取的a为: %d\n", a);
9     printf("获取的b为: %d\n", b);
10    printf("获取的c为: %d\n", c);
11
12    return EXIT_SUCCESS;
13 }
```

- 获取字符

```

1 // scanf 输入, 获取字符。
2 int main(void)
3 {
4     char ch1, ch2, ch3; // 一次性创建多个 char 变量
5
6     printf("请输入3个字符, 用空格隔分: ");
7
8     scanf("%c %c %c", &ch1, &ch2, &ch3);
9
10    printf("ch1 = %c\n", ch1);
11    printf("ch2 = %c\n", ch2);
12    printf("ch3 = %c\n", ch3);
13 }
```

```
14     return EXIT_SUCCESS;
15 }
```

注意事项:

1. **不要随意**在 scanf() 函数中, 添加 '\n'。如果添加了, 换行符, 会被scanf 当成 格式来约束用户输入。

1. printf() 中的 '\n' 用来向屏幕输出 换行。
2. scanf() 函数中的 '\n', 不能用来输出。因为这是一个输入函数。

2. 解决 VS中使用 scanf函数 报 C4996 错误:

以下是错误描述

C4996 'scanf': This function or variable may be unsafe. Consider using scanf_s instead. To disable deprecation, use _CRT_SECURE_NO_WARNINGS. See online help for details.
day04 C:\itcast\VSsource\day04\02-输出输出函数.c 43

3. 解决方法:

1. 在 .c 文件的 **第一行** 添加 `#define _CRT_SECURE_NO_WARNINGS`

getchar 函数

- 直接从键盘接收 一个字符, 并将得到的字符对应的ASCII返回。

- ```
1 int main(void)
2 {
3 char ch = getchar(); // 定义char 变量 ch, 接收 getchar() 函数返回值做为
 初值。
4
5 printf("ch的数值: %d\n", ch); // %hhd %hd
6 printf("ch的字符: %c\n", ch);
7
8 return EXIT_SUCCESS;
9 }
```

## 运算符

### 算数运算符

1. + - \* / : 先 乘除取余, 后加减。
2. 除法运算后, 得到的结果赋值给整型变量, 取整数部分。int c = 10/20; ---》 0
3. 除 0 : 错误操作, 不允许。printf("%d\n", 10/0);
4. 对0取余: 错误操作, 不允许。printf("%d\n", 123 % 0);
5. 不允许对小数取模。35 % 3.4;
6. 对负数取余, 结果为余数的绝对值。printf("%d\n", 10 % -3); ---> 1

### 自增自减运算符

- 前缀自增(++)、自减(--)

- 先自增、自减，再取值。

- ```
1 | int a = 10;
2 | ++a; // 等价于 a = a+1;
```

- 后缀自增(++), 自减(--)
 - 先取值，再自增、自减。

```
1 | // ++ / -- 运算符
2 | int main(void)
3 | {
4 |     // 前缀
5 |     int a = 10;
6 |     printf("%d\n", ++a); // ++a 等价于 a = a+1;
7 |
8 |     // 后缀
9 |     int b = 10;
10 |    printf("%d\n", b++); // b++ 等价于 b = b+1;
11 |
12 |    // 不管前缀、后缀、含有变量 ++/-- 表达式执行完后，变量均发生了自增、自减。
13 |    printf("b = %d\n", b);
14 |
15 |    return EXIT_SUCCESS;
16 | }
```

赋值运算符

1. “=”，在计算机中，只能完成“赋值”操作，一定是右边赋值给左边，也叫单向赋值符。
2. $a += 10$ // 等价于 $a = a + 10$;
3. $a -= 30$ // 等价于 $a = a - 30$;
4. $a \% = 5$; // 等价于 $a = a \% 5$;

比较运算符

- 真：1 (非0)，假：0
- == 判等符 (= 不是用来判等)
- != 不等
- < 小于
- <= 小于等于
- > 大于
- >= 大于等于
- 【强调】：数学运算中 $13 < \text{var} < 16$ 判断，在计算机中，要写成： $\text{var} > 13 \ \&\& \ \text{var} < 16$;

逻辑运算符

- 0 为假，非0 为真。(非0: 1、27、-9)
- 逻辑非(!)

- 非真为假，非假为真。

```
1 // 逻辑运算符
2 int main(void)
3 {
4     // 逻辑非
5     int a = 34; // 34 是非0，默认 a 为真。
6     int b = 0;
7
8     printf("a = %d\n", !a); // a为真，非a 为假!! ---> 0
9     printf("b = %d\n", !b); // b=0, b为假，非b, 为真。 ---> 1
10
11     return EXIT_SUCCESS;
12 }
13
```

- 逻辑与(&&)

- 同真为真，其余为假。

```
1 printf("=====%d\n", a && !b); // a为真，!b为真，真&&真 -- 真。 --->
1
```

- 逻辑或(||)

- 有真为真，同假为假。

```
1 printf("-----%d\n", a || !b); // a为真，!b为真，真||真 -- 真。 ---> 1
2     printf("-----%d\n", a || b); // a为真，b为假，真||假 --
    真。 ---> 1
3     printf("-----%d\n", !a || b); // !a为假，b为假，假||假 --
    假。 ---> 0
```

运算符优先级

[] () > ++/-- (后缀高于前缀) (强转)! (逻辑非) sizeof > 算数运算符(先乘除取余，后加减) > 比较运算符 > 逻辑运算符 > 三目运算 (条件运算) > 赋值运算符 > 逗号运算符。

- 一周左右的时间，记忆运算符优先级。

逗号运算符

```
1 int x, y, z;
2 int a = (x=1, y=2, z=3); // x=1, y=2, z=3 是一个逗号运算符表达式。运算结果为 a =
3     3;
4 int a = (x=1, z=3, y=27); // 运算结果为 a = 27;
```

- 含有“,”运算符表达式运算结果，是最后一个子表达式的结果。

三目运算符

- 语法：表达式1 ? 表达式2 : 表达式3
 - 表达式1 必须是一个判断表达式。
 - 结果为真：整个三目运算，返回表达式2
 - 结果为假：整个三目运算，返回表达式3

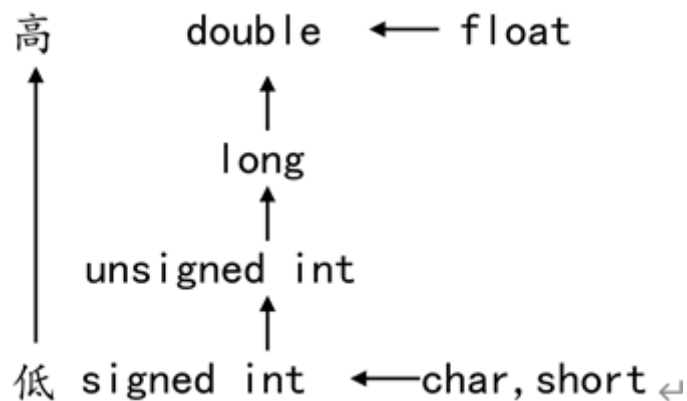
```
1 // 三目运算
2 int main(void)
3 {
4     int a = 40;
5     int b = 4;
6
7     //int m = a > b ? 69 : 10;
8     //printf("m = %d\n", m); // m = 69;
9
10    //int m = a < b ? 69 : 10;
11    //printf("m = %d\n", m); // m = 10;
12
13    // 三目运算支持嵌套
14    int m = a < b ? 69 : (a < b ? 3 : 5); // 先算表达式3，取5，整个三目运算，取表达式3--
    >5
15    printf("m = %d\n", m); // m = 5;
16
17    int m = a < b ? 69 : a < b ? 3 : 5;
18    printf("m = %d\n", m); // m = 5;
19
20    return EXIT_SUCCESS;
21 }
22
```

- 如果不使用 () ，三目运算默认的结合性，自右向左。

类型转换

隐式类型转换

1. 编译器自动完成。（小类型转大类型，同类型大小）



2. 由赋值产生。

- ```

1 int r = 3;
2 float s = 3.14 * r * r;
3 // 3.14 默认类型double, r 为int类型。运算过程中, 转换为 double 类型运算。
4 // 运算结束, 赋值给 s 时, 转换为 float。

```

- 小类型 ——> 大类型: 没问题。

- 大类型 ——> 小类型。可能丢失数据。

- VS中 Ctrl+F7 只编译, 检查语法错误, 不运行。

- ```

1 int main(void)
2 {
3     int a = 321;
4     char ch = a; // 用值为 321 的a变量, 给 char 类型赋值。
5
6     printf("ch = %d\n", ch); // 运行输出 65;
7
8     return EXIT_SUCCESS;
9 }

```

- 321: $2^8 = 256$ 有, 128 没有, 64 有, 32/16/8/4/2 没有, 有1

- 0000 0000 0000 0000 0000 0001 0100 0001 —— 321 二进制表现形式。

- 0000 0000 —— char 只有一个字节。

- 赋值后, char值为: 0100 0001 --- $1 + 64 = 65$

强制类型转换

- 语法:

1. 强转变量: (目标类型) 变量

2. 强转表达式: (目标类型) 表达式

```

1 int main(void)
2 {
3     float price = 3.6; // 单价
4     int weight = 4; // 斤数
5
6     //int sum = weight * (int)price; // 强转变量。
7     //printf("sum = %d\n", sum); // --- 12
8
9     int sum = (int)(weight * price); // 强转表达式。
10    printf("sum = %d\n", sum); // --- 14
11
12    return EXIT_SUCCESS;
13 }

```

if 分支语句

- if ... else 分支语句, 实现一种模糊匹配。匹配一个范围。

- if...else 分支

```

1  if (判断表达式) {
2      判别表达式为真，执行的代码。
3  }
4  else
5  {
6      判别表达式为假，执行的代码。
7  }
8  示例：
9      int a;
10
11     printf("请输入一个数: ");
12
13     int ret = scanf("%d", &a);
14
15     if (a > 0) {
16         printf("a > 0\n");
17     }
18     else
19     {
20         printf("a <= 0\n");
21     }

```

- 多个分支逻辑

```

1  if (判断表达式1) {
2      判别表达式1为真，执行的代码。
3  }
4  else if (判断表达式2)
5  {
6      判别表达式1为假，判断表达式2为真，执行的代码。
7  }
8  else if (判断表达式3)
9  {
10     判别表达式1为假，判断表达式2为假，判断表达式3为真，执行的代码。
11 }
12 ...
13 else
14 {
15     以上所有判别表达式都为假，执行的代码。
16 }
17 示例：
18     int score;
19
20     printf("请输入学生成绩: ");
21
22     scanf("%d", &score);
23
24     if (score >= 90)    // 优秀
25     {
26         printf("优秀\n");
27     }
28     else if (score >= 70 && score < 90)    // 70 < score < 90 错误写法。
29     {
30         printf("良好\n");
31     }
32     else if (score >= 60 && score < 70)

```

```

33     {
34         printf("及格\n");
35     }
36     else
37     {
38         printf("差劲\n");
39     }

```

练习:

- 编写程序，实现3只小猪秤体重。屏幕输入3个小猪的重量，借助if分支，找出最重的那只小猪。

```

1  int main(void)
2  {
3      int pig1, pig2, pig3;
4
5      printf("请输入3只小猪的重量: ");
6
7      scanf("%d %d %d", &pig1, &pig2, &pig3);
8
9      if (pig1 > pig2) // pig1 重
10     {
11         if (pig1 > pig3) // 1. 这行不能写分号。 2. 缩进使用 tab 实现
12         {
13             printf("第一只小猪最重, 体重为: %d\n", pig1);
14         }
15         else
16         {
17             printf("第3只小猪最重, 体重为: %d\n", pig3);
18         }
19     }
20     else // pig2 重
21     {
22         if (pig2 > pig3)
23         {
24             printf("第2只小猪最重, 体重为: %d\n", pig2);
25         }
26         else
27         {
28             printf("第3只小猪最重, 体重为: %d\n", pig3);
29         }
30     }
31     return EXIT_SUCCESS;
32 }
33 // 其他实现方法:
34 // 使用逻辑运算 if (pig1 > pig2 && pig1 > pig3) ---> pig1.
35 // 使用三目运算 pig1 > pig2 ? pig1 : pig2;

```

- 分支语句中，可以嵌套其他分支语句。
- else 总是找它前面最近的 未配对的 if 组合使用。

switch 分支语句

- 精确匹配。结构较清晰。较 if 语句执行效率略高。
- 缺点：不能直接判断区间，需要借助表达式运算。

```

1  switch(表达式)
2  {
3      case 1:
4          执行语句;
5          break;    // 表示一个分支执行结束。跳出 switch。
6      case 2:
7          执行语句;
8          break;    // 表示一个分支执行结束。跳出 switch。
9          .....
10     case N:
11         执行语句;
12         break;    // 表示一个分支执行结束。跳出 switch。
13     default:
14         其他情况，执行语句；（上述所有的case都不满足）
15         break;
16 }

```

- 练习：获取学生成绩，给出优良可差。

```

1  int main(void)
2  {
3      int score;
4
5      printf("请输入学生成绩: ");
6      scanf("%d", &score);
7
8      // 将 学生成绩，通过运算，得出可以放入 switch case 分支的 表达式。
9      switch (score/10)
10     {
11         case 10:
12             printf("优秀\n");
13             break;    // 表示当前分支结束。
14         case 9:
15             printf("优秀\n");
16             break;
17         case 8:
18             printf("良好\n");
19             break;
20         case 7:
21             printf("良好\n");
22             break;
23         case 6:
24             printf("及格\n");
25             break;
26         default:    // 所有case都不满足的其他情况。
27             printf("不及格");
28             break;
29     }
30     return EXIT_SUCCESS;
31 }

```

case穿透

- 一个case分支，如果没有 break；它执行完本case的代码后，会继续向下，执行下一个case 分支的代码。这称之为case穿透。
- 大多数情况下，一个case分支，应该对应一个 break；
- 利用case 穿透

```
1  switch (score/10)
2  {
3      case 10:          // 故意让 switch 发生 case穿透
4      case 9:
5          printf("优秀\n");
6          break;
7      case 8:          // 故意让 switch 发生 case穿透
8      case 7:
9          printf("良好\n");
10         break;
11     case 6:
12         printf("及格\n");
13         break;
14     default:          // 所有case都不满足的其他情况。
15         printf("不及格");
16         break;
17 }
```

while 循环语句

- 语法

- ```
1 while (判别表达式) // 如果为真，执行循环体，如果为假，跳出循环。
2 {
3 循环体
4 }
```

## 练习：

- 敲7：从 1~ 100 数数，逢 7 和 7 的倍数，敲桌子。
- 分析：
  - 7 的倍数：num % 7 == 0
  - 个位含7： num % 10 == 7
  - 十位含7： num / 10 == 7

```
1
2 int main(void)
3 {
4 //7的倍数: num % 7 == 0
5 //个位含7: num % 10 == 7
6 //十位含7: num / 10 == 7
7
8 int num = 1;
9
10 while (num <= 100)
```

```

11 {
12 // 判断敲桌子的时机
13 // if ((num % 7 == 0) || (num % 10 == 7) || (num / 10 == 7))
14 if (num % 7 == 0 || num % 10 == 7 || num / 10 == 7)
15 {
16 printf("敲桌子! \n");
17 }
18 else
19 {
20 printf("%d\n", num);
21 }
22 num++;
23 }
24 return EXIT_SUCCESS;
25 }

```

## do while 循环语句

- 无论如何先执行一次循环体，然后再判断循环是否应该继续。
- 语法：

```

1 do {
2 循环体
3 } while (判断表达式);

```

## 练习：

- 求水仙花数。一个三位数（100~999），各个位上数字的立方和等于本数字。
- 分析：
  - 3位数：100 ~ 999 —— 如：234、861
  - 个位数：int a = num % 10;
  - 十位数：int b = num / 10 % 10;
  - 百位数：int c = num / 100;

```

1 int main(void)
2 {
3 //-个位数: int a = num % 10;
4 //-十位数: int b = num / 10 % 10;
5 //-百位数: int c = num / 100;
6
7 int num = 100; // 数数从 100 开始。
8 int a, b, c; // 定义存储个位、十位、百位数 的变量。
9
10 do {
11 a = num % 10; // 求个位数。
12 b = num / 10 % 10;
13 c = num / 100;
14
15 // 判断 这个数字是否是“水仙花数”
16 if (a * a * a + b * b * b + c * c * c == num)
17 {

```

```
18 printf("水仙花数: %d\n", num);
19 }
20 num++; // 不断向后数数。
21
22 } while (num <= 999);
23
24 return EXIT_SUCCESS;
25 }
```